Laborator 3

Exerciții

1. Fie setul de date German Traffic Sign Recognition Benchmark (GTSRB), ce conține imagini color reprezentând indicatoare rutiere, sub diferite condiții de iluminare, stare meteorologică, sau poziție.

Afișați aleator o serie de eșantioane din setul de date, respectiv eticheta asociată.

Ce puteți spune cu privire la rezoluția imaginilor?

2. Afișați distribuția claselor în setul vostru de date (atât antrenare, cât și testare).

Câte clase distincte există în setul de date?

Ce puteți spune despre cele două distribuții?

Sugestie: puteți utiliza funcția *unique* implementată în biblioteca *numpy*.

3. Completați definiția instanței transform astfel încât să realizeze redimensionarea fiecărei imagini din setul de date la o rezoluție standard aleasă de dvs. (ex. 48 x 48 pixeli), să realizeze scalarea imaginii în intervalul [0, 1], respectiv să transforme imaginile în torch.FloatTensor. Iterați prin noul set de date și verificați efectul transformării.

Sugestie: consultați documentația modulului *transforms*.

4. Implementați o rețea convoluțională simplă conform Fig. 1, apoi afișați structura detaliată utilizând funcția summary din biblioteca torchsummary :

a. Rețeaua trebuie să conțină doar straturi convoluționale 2D (nn.Conv2d), straturi de agregare (ex. MaxPool2d, AvgPool2d), activare (ex. F.relu, F.sigmoid, etc sau nn.ReLU, nn.Sigmoid, etc), respectiv straturi complet conectate (Linear). **Sugestie**: vezi torch.nn.

b. Rețeaua trebuie să conțină cel puțin 3 straturi convoluționale, respectiv cel puțin 1 strat complet conectat.

c. Intrarea rețelei este un tensor de dimensiunea (B, C, H, W), unde B reprezintă dimensiunea batch-ului, C reprezintă dimensiunea canalelor de culoare, iar H și W reprezintă dimensiunile imaginii.

d. Ieșirea rețelei este un tensor de dimensiune (B, 10) unde B este dimensiunea batch-ului, iar n_classes reprezintă numărul de clase (vezi exercițiul 2).

e. Pentru a determina dimensiunea ieșirii fiecărui strat convoluțional Conv2d, sau MaxPool2d, se poate folosi formula:

$$D_{\text{out}} = \frac{D_{\text{in}} + 2 \cdot \text{padding} - \text{kernel_size}}{\text{stride}} + 1$$

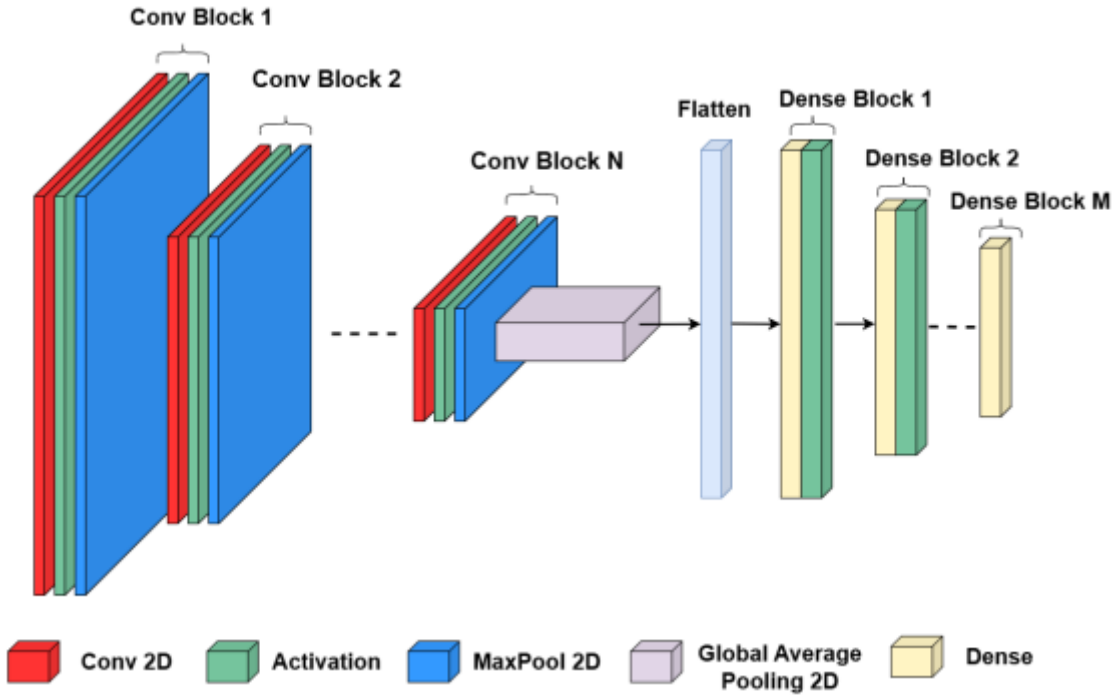


Fig.1 - Exemplificarea arhitecturii neurale convoluționale

5. Antrenați modelul și observați rezultatele. Cum influențează diverși hiperparametri performanța modelului (e.g. numărul de straturi convoluționale/dense, numărul de filtre convoluționale utilizate per strat, alegerea funcțiilor de activare, etc.)?

6. Adăugați straturi de normalizare a activărilor per lot (vezi nn.BatchNorm2d) și/sau straturi de regularizare (e.g. nn.Dropout) și reantrenați rețeaua. Cum este influențată performanța modelului?

7. Antrenați un model convoluțional de cel mult 100k parametri antrenabili, care să obțină o acuratețe de cel puțin 95% pe datele de testare.